

Εργασία 13: Προσομοίωση event-driven δικτύου σε Python

Συνεργάτες:

Αλέξανδρος-Γεώργιος Χαλαμπάκης
ΑΜ:1100754

Παναγιώτης Παπανικολάου
ΑΜ:1104804

Σημαντικο:

Αρχείο python event-driven-6.py: Είναι ο κώδικας μέχρι το ερώτημα 6.

Αρχείο python event-driven-7.py: Είναι ο κώδικας για το ερώτημα 7.

Αρχείο python event-driven-8.py: Είναι ο κώδικας για το ερώτημα 8.

Η αλλαγή του Bandwidth και packet size γίνεται χειροκίνητα.

Η αναφορά έχει μέσα πολλές φωτογραφίες για αυτό βγαίνει 25 σελίδες την μειώσαμε όσο μπορούσαμε.

Ερώτημα 1&2:

Για να δημιουργίσουμε ένα απλό δίκτυο σε Python χρησιμοποιώντας τη βιβλιοθήκη Simpy, το δίκτυο θα πρέπει να αποτελείται από:

- Έναν υπολογιστή-πηγή (Source Host) που στέλνει πακέτα.
- Έναν δρομολογητή (Router) που λαμβάνει τα πακέτα και τα προωθεί.
- Έναν υπολογιστή-προορισμό (Destination Host) που λαμβάνει τα πακέτα.
- Δύο συνδέσεις (Links): μία από την πηγή στον δρομολογητή και μία από τον δρομολογητή στον προορισμό.

Κάθε σύνδεση θα πρέπει να έχει δύο βασικές ιδιότητες:

Bandwidth (Εύρος Ζώνης): Ο ρυθμός με τον οποίο μπορούν να μεταδοθούν τα δεδομένα (π.χ., σε bits ανά δευτερόλεπτο).

Propagation Delay (Καθυστερήση Διάδοσης): Ο χρόνος που χρειάζεται ένα bit για να ταξιδέψει από το ένα άκρο της σύνδεσης στο άλλο.

Για να μοντελοποιήσουμε τη ροή των πακέτων, θα ορίσουμε τρεις κύριες "οντότητες" που λειτουργούν ως processes στο SimPy:

1. SourceHost: Δημιουργεί και στέλνει πακέτα.
2. Router: Λαμβάνει πακέτα από τη μία πλευρά και τα προωθεί στην άλλη.
3. DestinationHost: Λαμβάνει τα πακέτα στον τελικό προορισμό.

Παράμετροι:

Στην αρχή του αρχείου, ομαδοποιούμε όλες τις παραμέτρους που καθορίζουν τη συμπεριφορά του δικτύου.

- BANDWIDTH_BPS: Ορίζει το εύρος ζώνης σε **bits** ανά δευτερόλεπτο.
- BANDWIDTH_BYTES_PER_SEC: Επειδή το μέγεθος των πακέτων είναι σε **Bytes**, μετατρέπουμε το bandwidth σε Bytes ανά δευτερόλεπτο (1 Byte = 8 bits) για να έχουμε συνέπεια στις μονάδες μέτρησης.
- PACKET_SIZE_BYTES: Το σταθερό μέγεθος κάθε πακέτου.
- PACKET_INTERVAL_S: Ο ρυθμός με τον οποίο ο source_host δημιουργεί νέα πακέτα.
- PROP_DELAY_LINK1_S & PROP_DELAY_LINK2_S: Οι καθυστερήσεις διάδοσης για τις δύο συνδέσεις του δικτύου μας (Πηγή -> Router και Router -> Προορισμός).
- SIM_TIME_S: Ο συνολικός χρόνος που θα διαρκέσει η προσομοίωση.

Η Κλάση Packet:

Αυτή η κλάση λειτουργεί σαν ένα "κουτί" που περιέχει τις πληροφορίες για κάθε πακέτο που ταξιδεύει στο δίκτυο.

- `time_created`: Καταγράφουμε τη χρονική στιγμή (`env.now`) που δημιουργήθηκε το πακέτο. Αυτή η πληροφορία είναι θεμελιώδης για να υπολογίσουμε στο τέλος τη συνολική καθυστέρηση.
- `size_bytes`: Το μέγεθος του πακέτου, απαραίτητο για τον υπολογισμό της καθυστέρησης μετάδοσης.
- `packet_id`: Ένας μοναδικός αριθμός που βοηθά να παρακολουθούμε κάθε πακέτο ξεχωριστά στα μηνύματα που τυπώνουμε.

Η Συνάρτηση `source_host` :

Αυτή η συνάρτηση προσομοιώνει τη λειτουργία του υπολογιστή που στέλνει δεδομένα.

- `while True::` Είναι ένας ατέρμονος βρόχος που εξασφαλίζει ότι ο πομπός θα δημιουργεί πακέτα καθ' όλη τη διάρκεια της προσομοίωσης.
- `link_to_router.put(packet)`: Αυτή είναι η εντολή "αποστολής". Τοποθετεί το αντικείμενο `packet` μέσα στο `simpy.Store` (το κανάλι επικοινωνίας), καθιστώντας το διαθέσιμο για τον επόμενο κόμβο (τον `router`).
- `yield env.timeout(packet_interval)`: Αυτή είναι η πιο σημαντική εντολή του `SimPy`. "Παγώνει" την εκτέλεση της συνάρτησης για ένα συγκεκριμένο χρονικό διάστημα. Έτσι, μοντελοποιούμε τον ρυθμό αποστολής πακέτων.

Η Συνάρτηση `router`:

Μοντελοποιεί τις βασικές λειτουργίες ενός δρομολογητή.

- `packet = yield link_from_source.get()`: Η συνάρτηση "μπλοκάρει" σε αυτό το σημείο και περιμένει μέχρι να εμφανιστεί ένα πακέτο στο κανάλι εισόδου. Μόλις ο `source_host` κάνει `put`, ο `router` το παραλαμβάνει.
- **Καθυστέρηση Μετάδοσης (Transmission Delay):**

- `transmission_delay = packet.size_bytes / bandwidth_bytes_per_sec`: Εφαρμόζει τον τύπο Μέγεθος / Bandwidth για να βρει πόσος χρόνος χρειάζεται για να "μπουν" όλα τα bits του πακέτου στη γραμμή.
 - `yield env.timeout(transmission_delay)`: Προσομοιώνει αυτή την καθυστέρηση, κατά την οποία ο router είναι "απασχολημένος" με τη μετάδοση αυτού του πακέτου.
- **Καθυστέρηση Διάδοσης (Propagation Delay):**
 - `yield env.timeout(prop_delay_s)`: Αφού ολοκληρωθεί η μετάδοση, προσομοιώνει τον χρόνο που χρειάζεται το σήμα για να διασχίσει φυσικά το καλώδιο μέχρι τον επόμενο κόμβο.
- `link_to_dest.put(packet)`: Αφού ολοκληρωθούν οι καθυστερήσεις, προωθεί το πακέτο στο επόμενο κανάλι, προς τον προορισμό.

Η Συνάρτηση `destination_host`:

Προσομοιώνει τον τελικό παραλήπτη των πακέτων.

- `packet = yield link_from_router.get()`: Περιμένει να λάβει ένα πακέτο από τον router.
- `yield env.timeout(prop_delay_s)`: Προσομοιώνει την καθυστέρηση διάδοσης της *δεύτερης* σύνδεσης (Router -> Destination).
- `end_to_end_delay = env.now - packet.time_created`: Αυτός είναι ο τελικός υπολογισμός. Η τρέχουσα ώρα (`env.now`) είναι η ώρα άφιξης. Αφαιρώντας την ώρα δημιουργίας (`packet.time_created`), βρίσκουμε τον συνολικό χρόνο που το πακέτο πέρασε μέσα στο δίκτυο.

Main:

Αυτό το τμήμα "στήνει" και εκτελεί την προσομοίωση.

- `env = simpy.Environment()`: Δημιουργεί το κεντρικό αντικείμενο περιβάλλοντος που διαχειρίζεται τον χρόνο και τα γεγονότα.

- `link_... = simpy.Store(env)`: Δημιουργεί τα κανάλια επικοινωνίας (τις συνδέσεις) μεταξύ των κόμβων.
- `env.process(...)`: Δηλώνει στο περιβάλλον ποιες συναρτήσεις-διαδικασίες θέλουμε να τρέχουν ταυτόχρονα (τον πομπό, τον δρομολογητή και τον δέκτη).
- `env.run(until=SIM_TIME_S)`: Ξεκινά την προσομοίωση και την αφήνει να εξελιχθεί μέχρι ο εσωτερικός της χρόνος να φτάσει την τιμή `SIM_TIME_S`.

Συμπεράσματα:

Παρατηρούμε ότι η καθυστέρηση (End-to-End Delay) για το Πακέτο 1 είναι μεγαλύτερη από την καθυστέρηση για το Πακέτο 0. Αυτό συμβαίνει λόγω της διαφοράς μεταξύ του ρυθμού άφιξης και του ρυθμού μετάδοσης: Ο Router χρειάζεται 8 ms για να εκπέμψει (Transmission Delay) το Πακέτο 0 (1000 Bytes/1 Mbps). Η πηγή όμως στέλνει το επόμενο πακέτο (Πακέτο 1) μόλις 5 ms αργότερα. Ως αποτέλεσμα, όταν το Πακέτο 1 φτάνει στον Router, αυτός είναι ακόμα απασχολημένος με την εκπομπή του Πακέτου 0. Έτσι, το Πακέτο 1 αναγκάζεται να περιμένει, παρόλο που δεν έχουμε ορίσει ακόμα ρητά όριο στην ουρά. Αυτή η καθυστέρηση αναμονής (queuing delay) σηματοδοτεί την έναρξη της συμφόρησης στο δίκτυο, καθώς ο κόμβος δεν προλαβαίνει να εξυπηρετήσει τα πακέτα με τον ρυθμό που φτάνουν.

Ερώτημα 3

Για την υλοποίηση του τρίτου ερωτήματος, ο στόχος είναι να μετατραπεί ο δρομολογητής από έναν απλό κόμβο προώθησης σε έναν πιο ρεαλιστικό, ο οποίος διαθέτει μια **ουρά** αναμονής περιορισμένου μεγέθους. Όταν αυτή η ουρά γεμίζει, κάθε νέο πακέτο που φτάνει πρέπει να απορρίπτεται.

Νέες Παράμετροι:

Προστέθηκε μία νέα παράμετρος για τον καθορισμό της χωρητικότητας της ουράς:

- **QUEUE_SIZE = 10**: Αυτή η σταθερά ορίζει ότι η ουρά του δρομολογητή μπορεί να χωρέσει ταυτόχρονα μέχρι 10 πακέτα.

Επίσης, τροποποιήθηκε η παράμετρος `PACKET_INTERVAL_S` για να προκληθεί συμφόρηση:

- **`PACKET_INTERVAL_S = 0.005`**: Ο χρόνος άφιξης των πακέτων (5ms) ορίστηκε μικρότερος από τον χρόνο που απαιτείται για τη μετάδοσή τους (8ms). Αυτή η συνθήκη είναι απαραίτητη για να διασφαλιστεί ότι η ουρά θα γεμίσει και θα παρατηρήσουμε το φαινόμενο της απόρριψης πακέτων.

Μετατροπή Router σε κλάση:

Για τη σωστή μοντελοποίηση της ουράς, ο Router μετατράπηκε σε κλάση Python. Αυτή η προσέγγιση προσφέρει δύο βασικά πλεονεκτήματα:

1. Διατήρηση Κατάστασης: Η κλάση μπορεί να "θυμάται" την κατάστασή της, όπως το περιεχόμενο της ουράς και τον αριθμό των πακέτων που έχουν απορριφθεί.
2. Διαχωρισμός Λειτουργιών: Μας επιτρέπει να μοντελοποιήσουμε τις ταυτόχρονες λειτουργίες του router (λήψη, διαχείριση άφιξης, και αποστολή) ως ξεχωριστές μεθόδους που εκτελούνται παράλληλα.

Η κλάση Router περιλαμβάνει:

- `__init__(self, ...)`: Ο κατασκευαστής της κλάσης. Εδώ αρχικοποιείται η ουρά ως ένα `simpy.Store` με τη μέγιστη χωρητικότητα που ορίσαμε (`capacity=QUEUE_SIZE`). Επίσης, αρχικοποιείται ένας μετρητής `self.packets_dropped = 0`.
- `receive_packets(self, ...)`: Αυτή η μέθοδος λειτουργεί πλέον ως "υποδοχή". Ο ρόλος της είναι να λαμβάνει πακέτα από τη σύνδεση χωρίς να μπλοκάρει. Μόλις λάβει ένα πακέτο, ξεκινά αμέσως μια νέα, ανεξάρτητη διαδικασία (`handle_arrival`) για αυτό, επιτρέποντάς της να συνεχίσει να λαμβάνει τα επόμενα πακέτα που φτάνουν.
- `handle_arrival(self, packet)`: Αυτή είναι η νέα, κρίσιμη μέθοδος που μοντελοποιεί σωστά την άφιξη ενός πακέτου. Πρώτα, προσομοιώνει την καθυστέρηση διάδοσης της πρώτης σύνδεσης (`yield self.env.timeout(prop_delay_link1)`). Μόνο αφού περάσει αυτός ο χρόνος, το πακέτο θεωρείται ότι έχει φτάσει πραγματικά στον router. Τότε, εκτελείται ο έλεγχος:
 - Αν υπάρχει χώρος στην ουρά: Το πακέτο τοποθετείται σε αυτήν.
 - Αν η ουρά είναι γεμάτη: Το πακέτο απορρίπτεται και ο μετρητής `packets_dropped` αυξάνεται.

Βελτιωμένη Λειτουργία Αποστολής: Στη νέα υλοποίηση της κλάσης Router, διορθώθηκε ο τρόπος διαχείρισης των καθυστερήσεων ώστε να επιτρέπεται η παραλληλία (pipelining). Συγκεκριμένα, στη μέθοδο `send_packets`:

1. **Transmission Delay:** Ο Router απασχολείται (μένει blocked) μόνο για το χρονικό διάστημα που απαιτείται για την εκπομπή των δεδομένων (`packet_size / bandwidth`).
2. **Propagation Delay:** Μόλις ολοκληρωθεί η εκπομπή, το πακέτο "παραδίδεται" στο μέσο μεταφοράς. Χρησιμοποιείται η βοηθητική διαδικασία `propagate_output` (μέσω `env.process`), η οποία προσομοιώνει τον χρόνο ταξιδιού του πακέτου στο καλώδιο ανεξάρτητα.

Με αυτόν τον τρόπο, ο Router ελευθερώνεται άμεσα μετά την εκπομπή (transmission) και μπορεί να εξυπηρετήσει το επόμενο πακέτο από την ουρά, ενώ το προηγούμενο πακέτο ταξιδεύει ακόμα προς τον προορισμό.

Ανάλυση Αποτελεσμάτων του Ερωτήματος 3:

Τα αποτελέσματα της προσομοίωσης επιβεβαιώνουν την ορθή λειτουργία του βελτιωμένου μοντέλου ουράς υπό συνθήκες φόρτου.

- **Το Γέμισμα της Ουράς (Queue Buildup):**

Ο ρυθμός άφιξης των πακέτων είναι σταθερός στο 1 πακέτο κάθε 5ms (200 pkts/s). Με το Bandwidth περιορισμένο στο 1 Mbps, ο χρόνος που απαιτείται για την εκπομπή (transmission delay) ενός πακέτου 1000 Bytes είναι 8ms (που αντιστοιχεί σε μέγιστο ρυθμό εξυπηρέτησης 125 pkt/s).

Επειδή ο ρυθμός άφιξης είναι μεγαλύτερος από τον ρυθμό εξυπηρέτησης ($200 > 125$), η ουρά αρχίζει να γεμίζει συστηματικά αμέσως μόλις φτάσουν τα πρώτα πακέτα, καθώς ο router δεν προλαβαίνει να "αδειάσει" το buffer του.

- **Η Πρώτη Απόρριψη (Packet Drop):**

Μόλις η ουρά φτάσει στη μέγιστη χωρητικότητά της (10 πακέτα), το σύστημα αδυνατεί να αποθηκεύσει νέα δεδομένα. Κάθε επόμενο πακέτο που φτάνει στον router και βρίσκει την ουρά πλήρη, απορρίπτεται οριστικά. Λόγω της μεγάλης διαφοράς στους ρυθμούς (άφιξη vs αναχώρηση), αυτό το φαινόμενο εμφανίζεται πολύ γρήγορα στην αρχή της προσομοίωσης.

- **Η Κατάσταση Συμφόρησης (Congestion):**

Το σύστημα εισέρχεται σε κατάσταση μόνιμου κορεσμού. Παρόλο που η νέα υλοποίηση (pipelining) επιτρέπει στον router να λειτουργεί αποδοτικά και να λαμβάνει πακέτα ενώ ταυτόχρονα μεταδίδει, το Bandwidth του 1 Mbps αποτελεί το φυσικό όριο (bottleneck). Η ουρά παραμένει συνεχώς γεμάτη και το ποσοστό των απορρίψεων σταθεροποιείται, αντανakλώντας ακριβώς την περίσσεια των πακέτων που δεν χωρούν στο κανάλι.

Ερωτημα 4:

Μεθοδολογία Υλοποίησης

Για τη συλλογή των δεδομένων, δημιουργήθηκαν κατάλληλες δομές (λίστες και λεξικά) οι οποίες ενημερώνονται δυναμικά κατά την εκτέλεση της προσομοίωσης.

- **End-to-End Delay:** Η συνάρτηση `destination_host` επεκτάθηκε ώστε, για κάθε πακέτο που λαμβάνει, να υπολογίζει τη συνολική καθυστέρηση ως τη διαφορά μεταξύ του τρέχοντος χρόνου προσομοίωσης (`env.now`) και του χρόνου δημιουργίας του πακέτου (`packet.time_created`). Κάθε υπολογιζόμενη καθυστέρηση αποθηκεύεται μαζί με τον χρόνο άφιξης σε μια λίστα (`delay_stats`) για μετέπειτα οπτικοποίηση.
- **Packet Loss Rate:**
 - Ο αριθμός των πακέτων που απορρίπτονται καταγράφεται ήδη από την κλάση Router στη μεταβλητή `self.packets_dropped`.
 - Για τον υπολογισμό του ποσοστού, χρειαζόμαστε και τον συνολικό αριθμό των πακέτων που στάλθηκαν. Για τον λόγο αυτό, προστέθηκε ένας μετρητής (`packets_sent_stats`) που ενημερώνεται από τη συνάρτηση `source_host` κάθε φορά που δημιουργείται ένα πακέτο.
 - Ο ρυθμός απώλειας υπολογίζεται στο τέλος της προσομοίωσης με τον τύπο: $(\text{πακέτα που απορρίφθηκαν} / \text{σύνολο πακέτων που στάλθηκαν}) * 100$.
- **Throughput (Ρυθμαπόδοση):**
 - Η μετρική αυτή αντιπροσωπεύει τον πραγματικό ρυθμό μετάδοσης χρήσιμων δεδομένων που φτάνουν στον προορισμό.
 - Στη συνάρτηση `destination_host`, κάθε φορά που λαμβάνεται ένα πακέτο, ο συνολικός όγκος των ληφθέντων δεδομένων (σε bits) αυξάνεται (`received_stats['total_bits']`).

- Ταυτόχρονα, καταγράφεται ο χρόνος άφιξης του τελευταίου πακέτου (`received_stats['last_arrival_time']`).
- Το throughput υπολογίζεται στο τέλος ως: (Σύνολο bits που λήφθηκαν) / (Χρόνος άφιξης του τελευταίου πακέτου).
- **Queue Size Monitoring:**
 - Για την παρακολούθηση του μεγέθους της ουράς, δημιουργήθηκε μια νέα, ανεξάρτητη διεργασία (process) της SimPy, η `monitor_queue_size`.
 - Αυτή η συνάρτηση εκτελείται παράλληλα με τις υπόλοιπες και, σε τακτά χρονικά διαστήματα (`QUEUE_MONITOR_INTERVAL_S`), "φωτογραφίζει" το τρέχον μέγεθος της ουράς του router (`len(router.queue.items)`) και αποθηκεύει το ζεύγος (χρόνος, μέγεθος ουράς) σε μια λίστα (`queue_size_stats`).

Μετά το τέλος της προσομοίωσης, χρησιμοποιείται η βιβλιοθήκη `matplotlib` για τη δημιουργία δύο γραφικών παραστάσεων που απεικονίζουν τα δεδομένα που συλλέχθηκαν:

1. **Γράφημα End-to-End Delay:** Παρουσιάζει την καθυστέρηση κάθε πακέτου σε σχέση με τον χρόνο άφιξής του στον προορισμό.
2. **Γράφημα Queue Size:** Δείχνει πώς μεταβάλλεται το μέγεθος της ουράς του router κατά τη διάρκεια της προσομοίωσης.

3. Ανάλυση Αποτελεσμάτων

- Τα αποτελέσματα της προσομοίωσης για το σενάριο του 1 Mbps αποτυπώνουν ξεκάθαρα το φαινόμενο της συμφόρησης (network congestion), αλλά με βελτιωμένη απόδοση σε σχέση με την αρχική (blocking) προσέγγιση.

Μαθηματική Εξήγηση: Ρυθμός Άφιξης: $1 \text{ πακετο} / 0.005 \text{ s} = 200 \text{ πακετα/s}$.

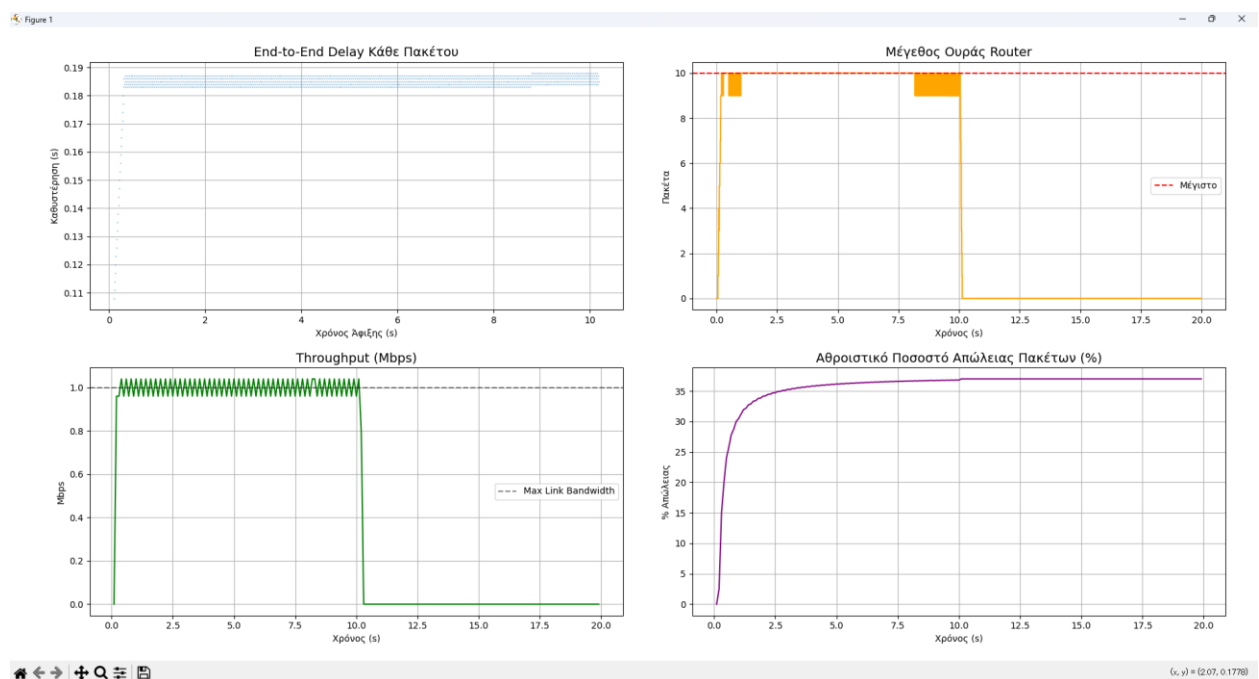
Ρυθμός Εξυπηρέτησης (Transmission Rate): Ο router απασχολείται μόνο για τη μετάδοση. $8000 \text{ bits} / 1.000.000 \text{ bps} = 0.008 \text{ s}$ (8 ms). Μέγιστος ρυθμός εξόδου = $1 / 0.008 = 125 \text{ πακετα/s}$. Επειδή $200 > 125$, η ουρά γεμίζει αναπόφευκτα.

Παρατηρήσεις στα Γραφήματα:

End-to-End Delay: Η καθυστέρηση αυξάνεται καθώς γεμίζει η ουρά και σταθεροποιείται περίπου στα 0.188s. Αυτό προκύπτει από το άθροισμα των σταθερών καθυστερήσεων ($0.05 + 0.008 + 0.05 = 0.108 \text{ s}$) συν την καθυστέρηση αναμονής στην πλήρη ουρά 10 πακέτων ($10 \times 0.008 = 0.08 \text{ s}$).

Packet Loss Rate: Το ποσοστό απώλειας σταθεροποιείται περίπου στο 37.5%. Αυτό εξηγείται θεωρητικά: Ο router απορρίπτει τη διαφορά μεταξύ ρυθμού άφιξης και εξυπηρέτησης ($200 - 125 = 75$ πακέτα χαμένα ανά δευτερόλεπτο). Ποσοστό: $75/200 = 37.5\%$.

Throughput: Σε αντίθεση με την παλιά υλοποίηση, το throughput τώρα φτάνει το 1.0 Mbps. Αυτό συμβαίνει διότι η ουρά είναι συνεχώς γεμάτη, οπότε ο σύνδεσμος χρησιμοποιείται στο 100% της χωρητικότητάς του.



Ερωτημα 5:

Μεθοδολογία:

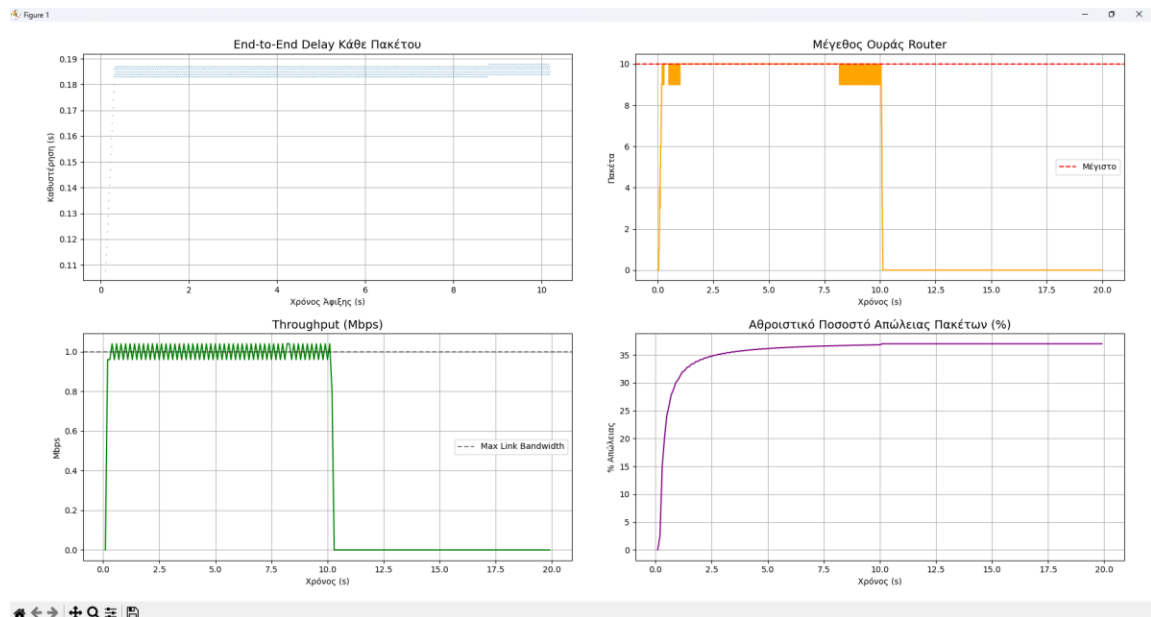
Εκτελέσαμε την προσομοίωση για τρεις διαφορετικές τιμές εύρους ζώνης (Bandwidth): 1 Mbps, 10 Mbps και 100 Mbps, διατηρώντας σταθερό τον ρυθμό άφιξης (1 πακέτο κάθε 5ms) και το μέγεθος πακέτου (1000 Bytes).

Ανάλυση Αποτελεσμάτων:

Σε αντίθεση με την αρχική απλοϊκή προσέγγιση, η σωστή μοντελοποίηση (pipelining) αναδεικνύει τον κρίσιμο ρόλο του Bandwidth.

1. Περίπτωση 1 Mbps:

- Transmission Delay: $8000 / 1.000.000 \text{ bps} = 8 \text{ ms}$
- Interval Πακέτων: 5ms .
- Παρατήρηση: Ο χρόνος που χρειάζεται ο Router για να στείλει ένα πακέτο (8ms) είναι μεγαλύτερος από τον ρυθμό που έρχονται τα νέα πακέτα (5ms).
- Αποτέλεσμα: Ο Router δεν προλαβαίνει. Η ουρά γεμίζει γρήγορα, έχουμε τεράστια καθυστέρηση και μεγάλο ποσοστό Packet Loss (όπως και στο Ερώτημα 4).



2. Περίπτωση 10 Mbps:

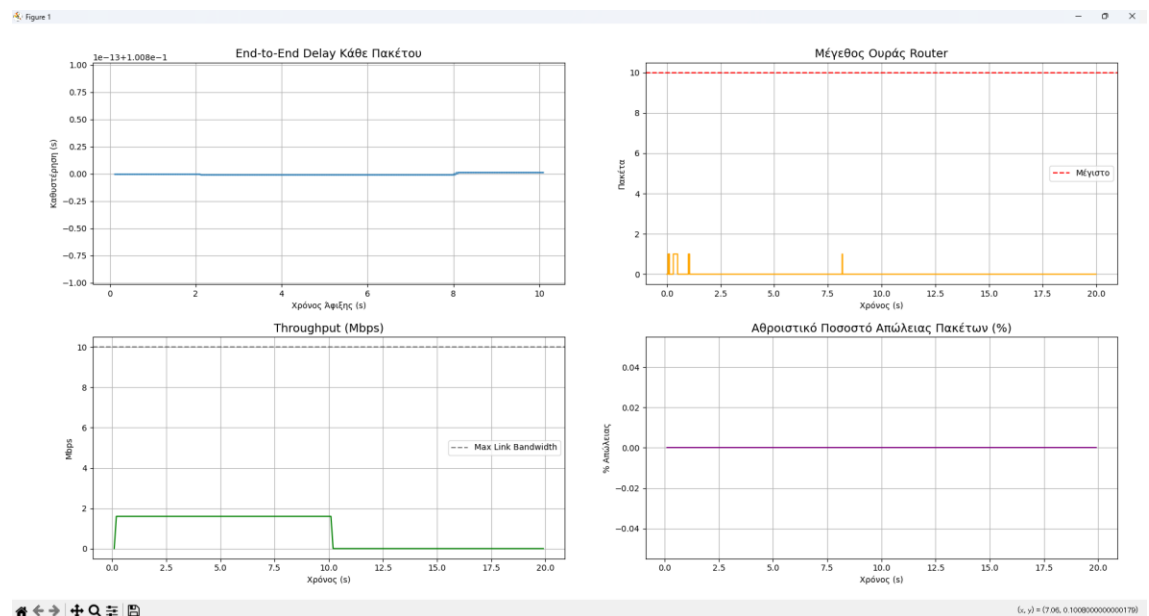
- Transmission Delay: $8000 / 10.000.000 \text{ bps} = 0.8 \text{ ms}$.
- Interval Πακέτων: 5 ms.
- Παρατήρηση: Τώρα, ο Router μπορεί να στείλει το πακέτο πολύ γρήγορα (0.8ms), πολύ πιο γρήγορα από τον ρυθμό που καταφθάνουν τα νέα (5 ms).
- Αποτέλεσμα:

Queue Size: Η ουρά παραμένει σχεδόν μόνιμα άδεια (0 πακέτα), καθώς κάθε πακέτο εξυπηρετείται άμεσα.

Delay: Η καθυστέρηση σταθεροποιείται στην ελάχιστη δυνατή τιμή (0.1008 s), που αντιστοιχεί μόνο στον χρόνο διάδοσης και μετάδοσης, χωρίς καθυστέρηση ουράς.

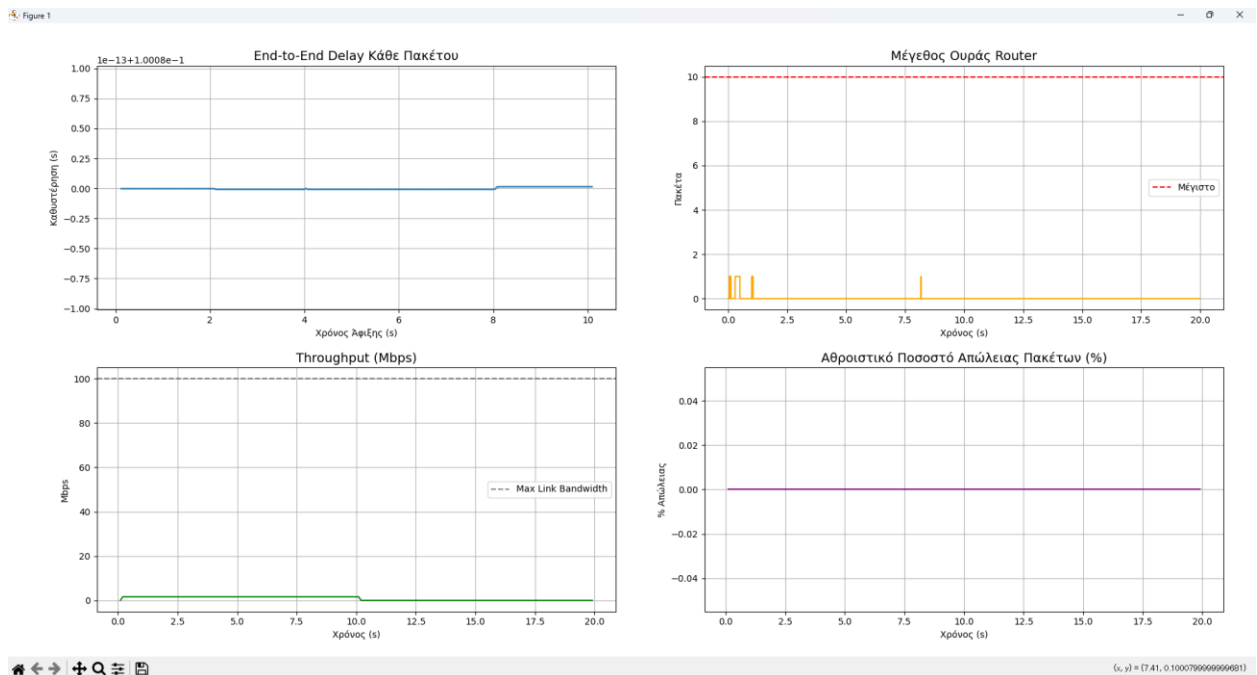
Loss Rate: Το ποσοστό απώλειας μηδενίζεται (0%).

Throughput: Σταθεροποιείται στα 1.6 Mbps. Παρόλο που το δίκτυο αντέχει 10 Mbps, το throughput καθορίζεται από τον ρυθμό αποστολής της πηγής ($200 \text{ pkt/s} \times 8000 \text{ bits/pkt} = 1.6 \text{ Mbps}$).



3. Περίπτωση 100 Mbps:

- **Transmission Delay:** $8000 \text{ bits} / 100.000.000 \text{ bps} = 0.08 \text{ ms}$.
- **Αποτέλεσμα:** Η εικόνα είναι παρόμοια με τα 10 Mbps. Η καθυστέρηση μετάδοσης γίνεται αμελητέα (0.08 ms). Δεν υπάρχει καμία ουσιαστική διαφορά στην απόδοση σε σχέση με τα 10 Mbps για τον συγκεκριμένο φόρτο, καθώς και στις δύο περιπτώσεις το δίκτυο υπερκαλύπτει τις απαιτήσεις της πηγής.



Ερώτημα 6:

Μεθοδολογία:

Για να μελετήσουμε πώς επηρεάζει το μέγεθος της πληροφορίας την απόδοση του δικτύου, εκτελέσαμε μια σειρά προσομοιώσεων μεταβάλλοντας το μέγεθος του πακέτου (Packet Size), διατηρώντας όμως σταθερές όλες τις άλλες παραμέτρους:

- **Bandwidth:** 1 Mbps (ώστε να υπάρχει πιθανότητα συμφόρησης).
- **Ρυθμός Άφιξης:** 1 πακέτο κάθε 5ms (200 πακέτα/δευτερόλεπτο).
- **Μεγέθη Πακέτων που εξετάστηκαν:** 100 Bytes, 1000 Bytes, 3000 Bytes, 5000 Bytes.

Θεωρητική Προσέγγιση (Φόρτος Δικτύου):

Η αλλαγή του μεγέθους πακέτου επηρεάζει άμεσα τον Ρυθμό Παραγωγής Δεδομένων (Offered Load) της πηγής.

$$\text{Load (bps)} = \text{Packet Size (bits)} \times \text{Arrival Rate (pkts/s)}$$

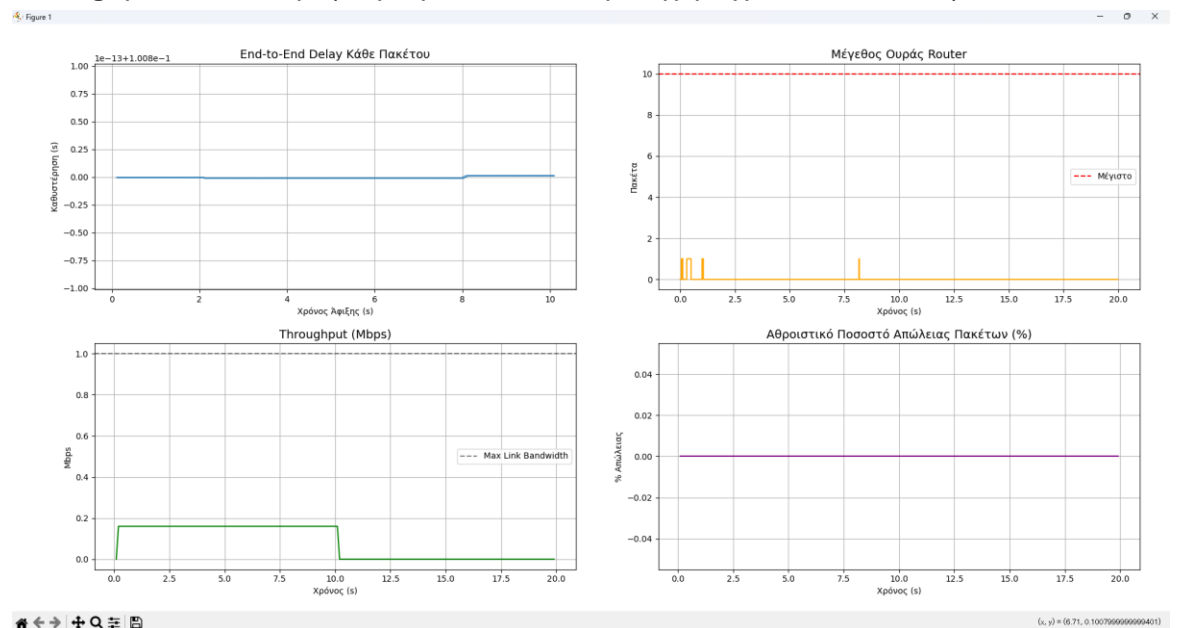
Συγκρίνοντας το Load με το Bandwidth (1 Mbps), μπορούμε να προβλέψουμε τη συμπεριφορά:

1. **100 Bytes:** $800\text{bits} \times 200 = 160.000 \text{ bps} = 0.16 \text{ Mbps}$ (Πολύ κάτω από το όριο).
2. **1000 Bytes:** $8000\text{bits} \times 200 = 1.6 \text{ Mbps}$ (Πάνω από το όριο άρα υπάρχει Συμφόρηση).
3. **3000 Bytes:** $24000\text{bits} \times 200 = 4.8 \text{ Mbps}$ (Έντονη Συμφόρηση).
4. **5000 Bytes:** $40000\text{bits} \times 200 = 8.0 \text{ Mbps}$ (Ακραία Συμφόρηση).

Ανάλυση Αποτελεσμάτων:

1. Μικρά Πακέτα (100 Bytes):

- **Συμπεριφορά:** Το δίκτυο υπολειτουργεί (underutilized). Ο Router χρειάζεται μόλις 0.8ms για να στείλει κάθε πακέτο, ενώ τα πακέτα έρχονται κάθε 5ms.
- **Μετρικές:**
 - Queue Size: 0 (Άδεια ουρά).
 - Packet Loss: 0%.
 - Throughput: 0.16 Mbps (Περιορίζεται από την πηγή, όχι από το δίκτυο).

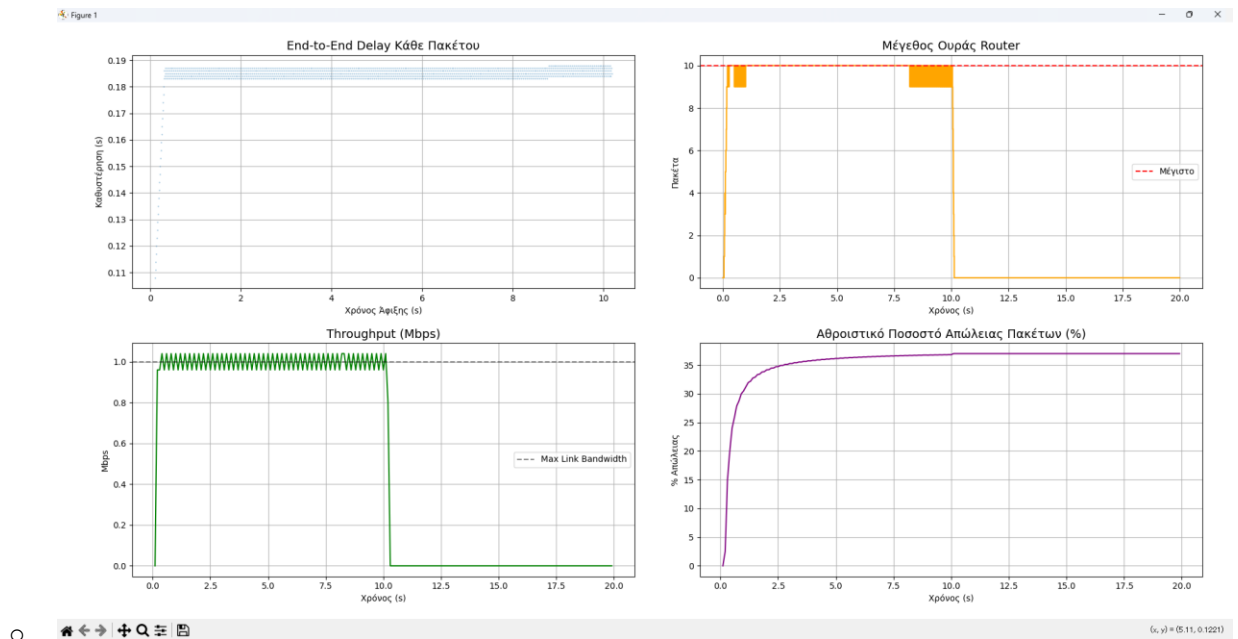


2. Μεσαία Πακέτα (1000 Bytes):

- **Συμπεριφορά:** Όπως αναλύθηκε στο Ερώτημα 4, το Load 1.6Mbps ξεπερνά το Bandwidth 1.0 Mbps.

- **Μετρικές:**

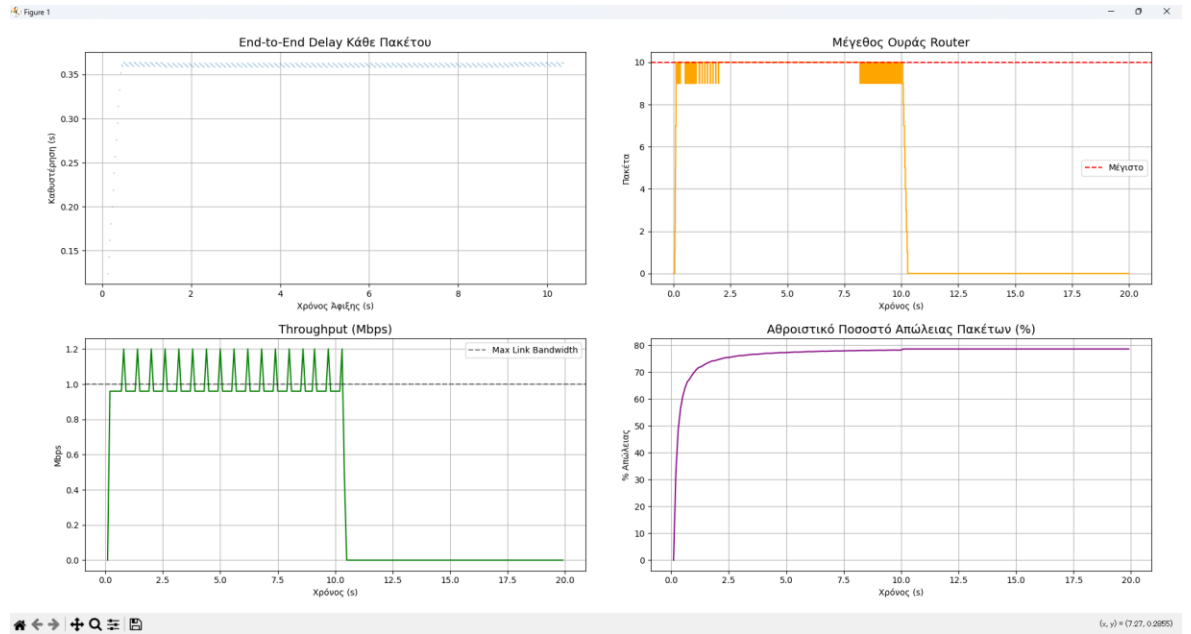
- Queue Size: Γεμάτη (10).
- Packet Loss: 37.5%.
- Throughput: 1.0 Mbps (Max capacity).



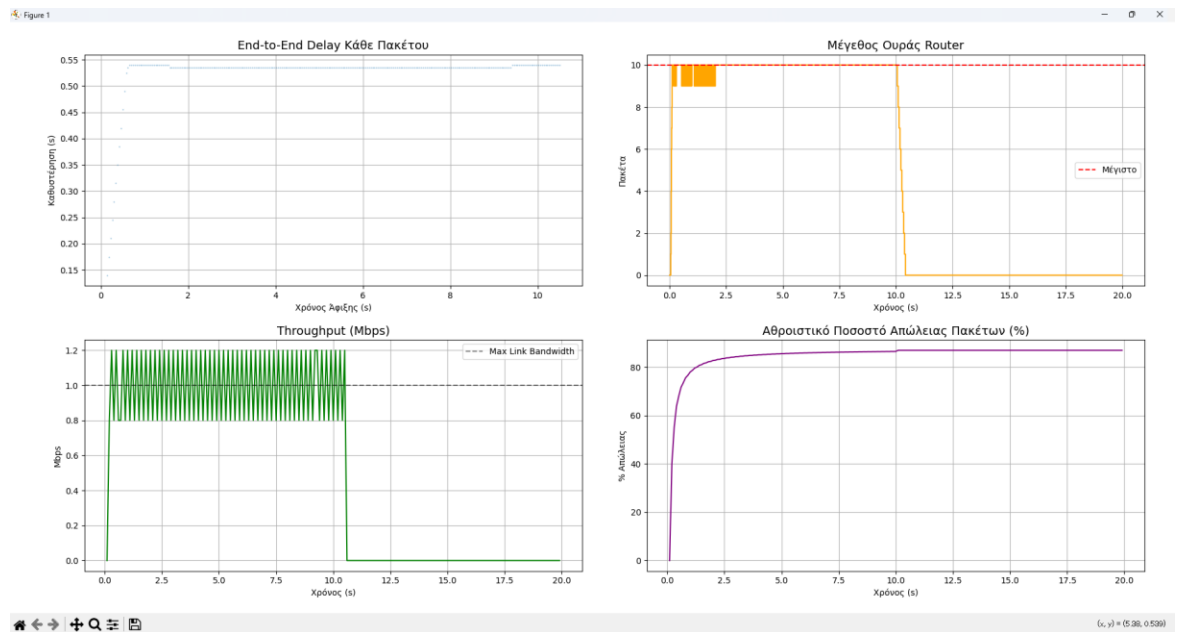
3. Μεγάλα (3000 Bytes) & Πολύ Μεγάλα Πακέτα (5000 Bytes):

- **Συμπεριφορά:** Η κατάσταση επιδεινώνεται δραματικά. Για πακέτο 5000 Bytes, ο Router χρειάζεται 40ms για να το στείλει (Transmission Delay), ενώ στο ίδιο διάστημα έχουν φτάσει στην ουρά άλλα 8 νέα πακέτα $40/5 = 8$.
- **Μετρικές:**
 - **Transmission Delay:** Αυξάνεται γραμμικά με το μέγεθος (φτάνει τα 40ms στα 5000 Bytes).
 - **Packet Loss:** Εκτοξεύεται. Στα 5000 Bytes, η πηγή στέλνει 8 Mbps σε κανάλι 1 Mbps, άρα χάνουμε περίπου το 87.5 των δεδομένων.
 - **Throughput:** Παραμένει "καρφωμένο" στο 1.0Mbps, καθώς το κανάλι είναι συνεχώς απασχολημένο.

3000:



5000:



Ερώτημα 7:

Μεθοδολογία και Τοπολογία

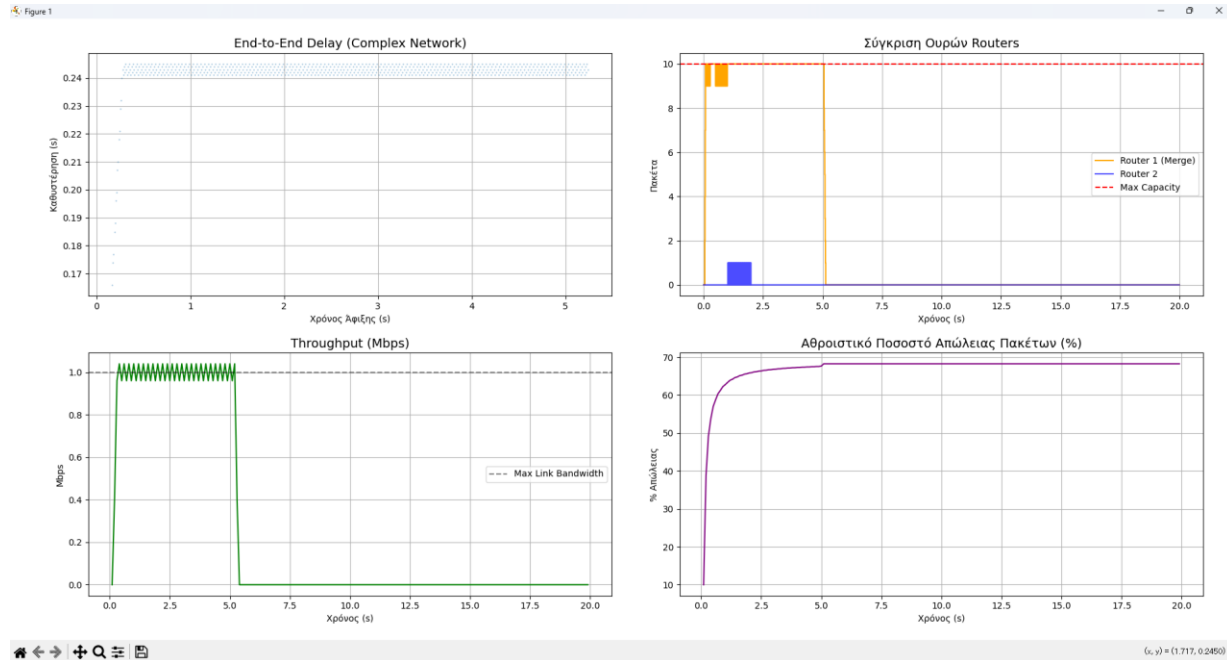
Για να εξετάσουμε την επίδραση της πολυπλοκότητας και της αύξησης των χρηστών, δημιουργήσαμε μια σύνθετη τοπολογία τύπου "Y" που αποτελείται από:

- **Hosts:** Δύο Πηγές (Host A, Host B) και έναν Προορισμό (Destination).
- **Routers:**
 - **Router 1 (Aggregator):** Συγκεντρώνει την κίνηση από τις δύο πηγές. Λειτουργεί ως το σημείο συγχώνευσης (Merge Point).
 - **Router 2 (Backbone):** Μεταφέρει την κίνηση από τον R1 στον προορισμό.
- **Σενάριο Κυκλοφορίας:** Κάθε Host στέλνει 1000 πακέτα (Σύνολο 2000) με τον ίδιο ρυθμό (1 πακέτο/5ms). Αυτό διπλασιάζει τον φόρτο εισόδου στον R1 σε σχέση με τα προηγούμενα ερωτήματα (400 pkts/s συνολικά).

2. Βασική Ανάλυση (Bandwidth 1 Mbps - 1000 Bytes)

Σε αυτό το σενάριο, το δίκτυο λειτουργεί υπό συνθήκες ακραίας συμφόρησης.

- **Συμπεριφορά Bottleneck (Router 1):** Ο Router 1 δέχεται 400 pkts/s αλλά μπορεί να εξυπηρετήσει μόνο 125 pkts/s (λόγω του 1 Mbps).
- **Αποτελέσματα:**
 - **Packet Loss:** Εκτοξεύτηκε στο **~68.25%**. Αυτό συμβαίνει επειδή ο Router 1 απορρίπτει πάνω από τα 2/3 των πακέτων που φτάνουν.
 - **Queue Size:** Η ουρά του Router 1 (Aggregator) παραμένει μόνιμα γεμάτη. Αντίθετα, η ουρά του Router 2 είναι άδεια, καθώς ο R1 "φιλτράρει" την κίνηση.
 - **Delay:** Η μέση καθυστέρηση αυξήθηκε στα **0.246s** λόγω των επιπλέον hops και της αναμονής στον R1.

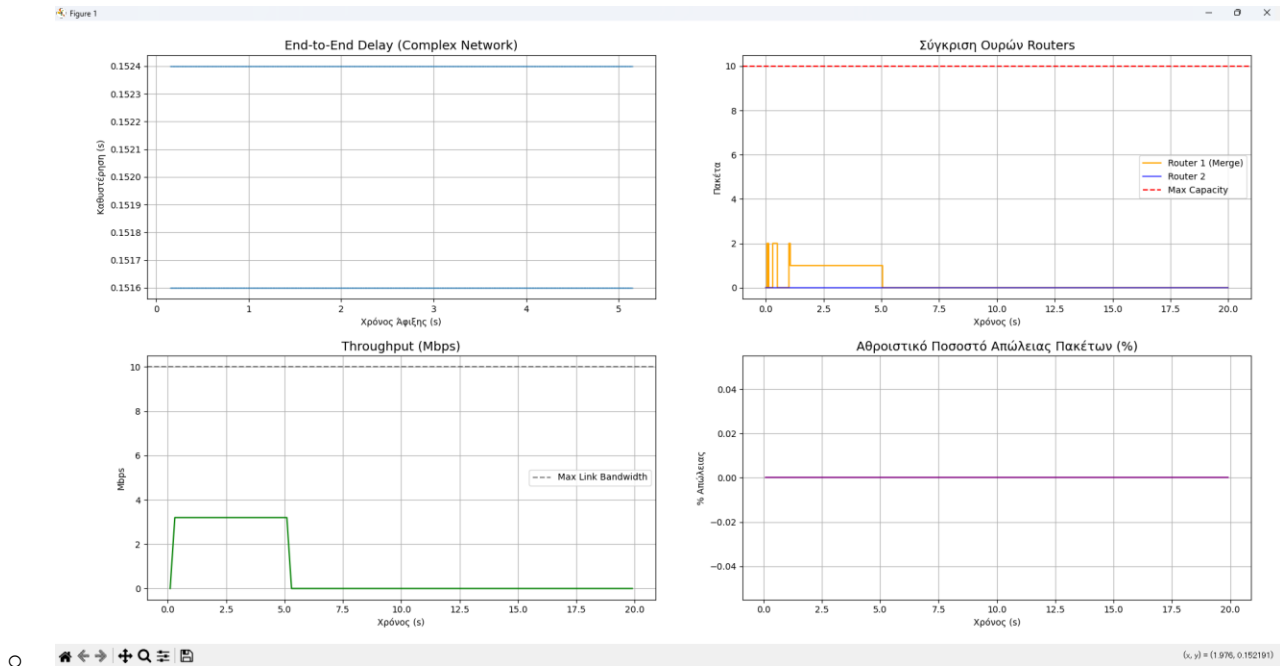


3. Ανάλυση Επίδρασης Bandwidth (10 Mbps & 100 Mbps)

Επανάλαβαμε το πείραμα αυξάνοντας τη χωρητικότητα των συνδέσεων, για να δούμε αν η πολυπλοκότητα του δικτύου εξακολουθεί να δημιουργεί προβλήματα.

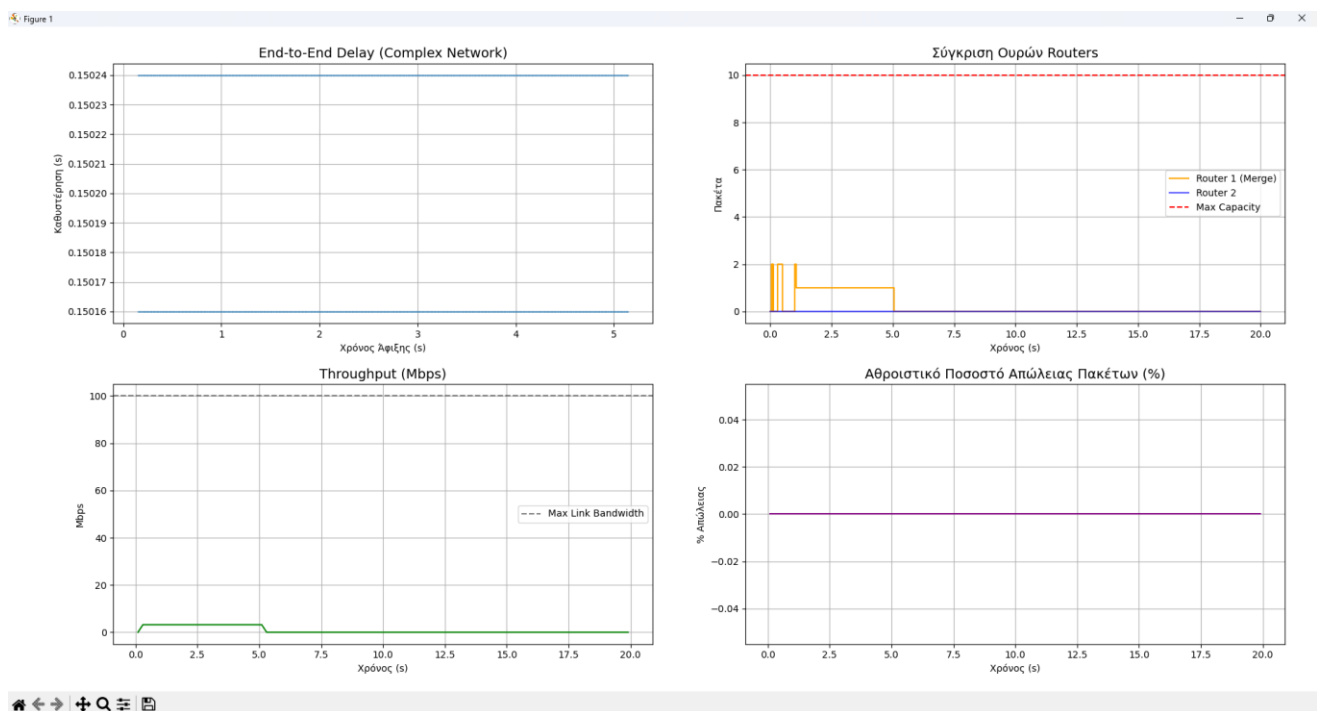
A. Bandwidth 10 Mbps

- **Load vs Capacity:** Το συνολικό Load είναι $400 \text{ pkts} \times 8000 \text{ bits} = 3.2 \text{ Mbps}$. Το κανάλι έχει 10 Mbps.
- **Παρατήρηση:** Επειδή $3.2 < 10$, το δίκτυο πλέον **αντέχει** τον διπλασιασμένο φόρτο.
- **Αποτελέσματα:**
 - **Loss:** Μηδενικό (0%).
 - **Queue:** Και οι δύο ουρές (R1 & R2) παραμένουν άδειες.
 - **Delay:** Μειώνεται στο ελάχιστο δυνατό (propagation + transmission για 3 hops).



B. Bandwidth 100 Mbps

- Παρατήρηση:** Όπως και στο απλό δίκτυο, η αύξηση στα 100 Mbps δεν προσφέρει ορατή βελτίωση σε σχέση με τα 10 Mbps, καθώς το δίκτυο ήταν ήδη επαρκές. Το throughput περιορίζεται από τις πηγές στα 3.2 Mbps.

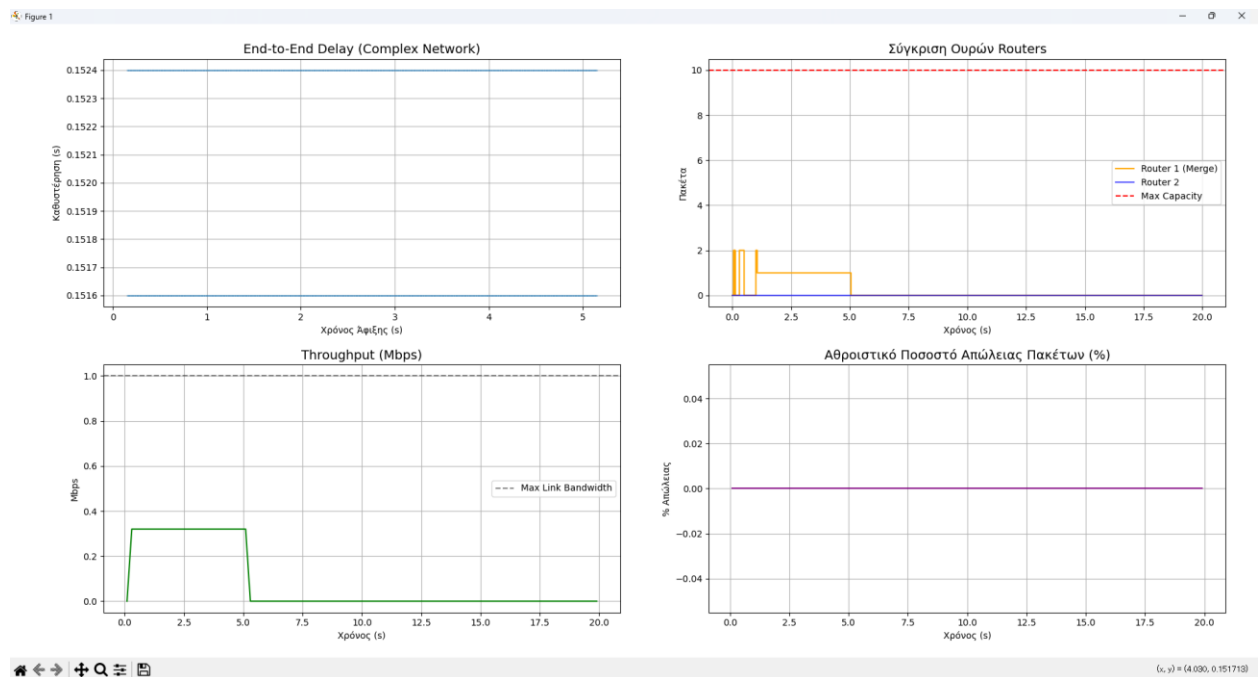


4. Ανάλυση Επίδρασης Μεγέθους Πακέτου (1 Mbps)

Διατηρώντας το Bandwidth στο 1 Mbps, εξετάσαμε πώς αντιδρά η σύνθετη τοπολογία σε διαφορετικά μεγέθη πακέτων.

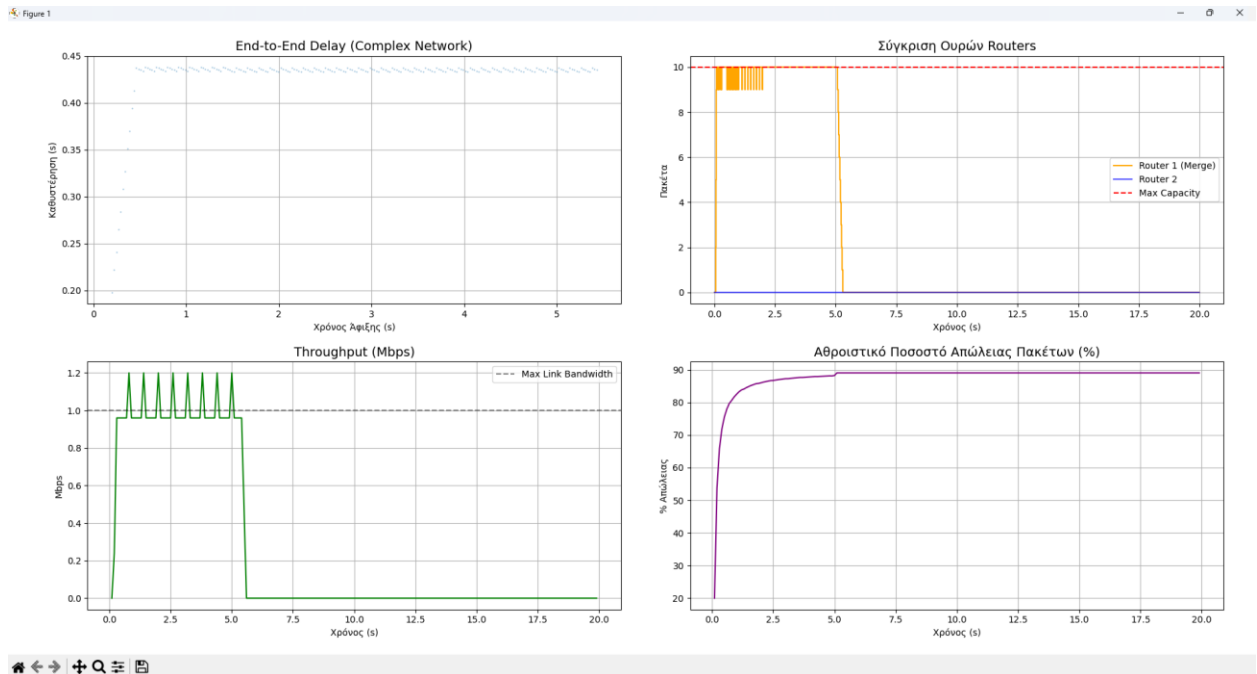
A. Μικρά Πακέτα (100 Bytes)

- **Load:** $400 \text{ pkts/s} \times 800 \text{ bits} = 0.32 \text{ Mbps}$.
- **Αποτέλεσμα:** Το δίκτυο λειτουργεί άνετα ($0.32 < 1.0$). Δεν υπάρχει συμφόρηση, παρόλο που έχουμε δύο πηγές.



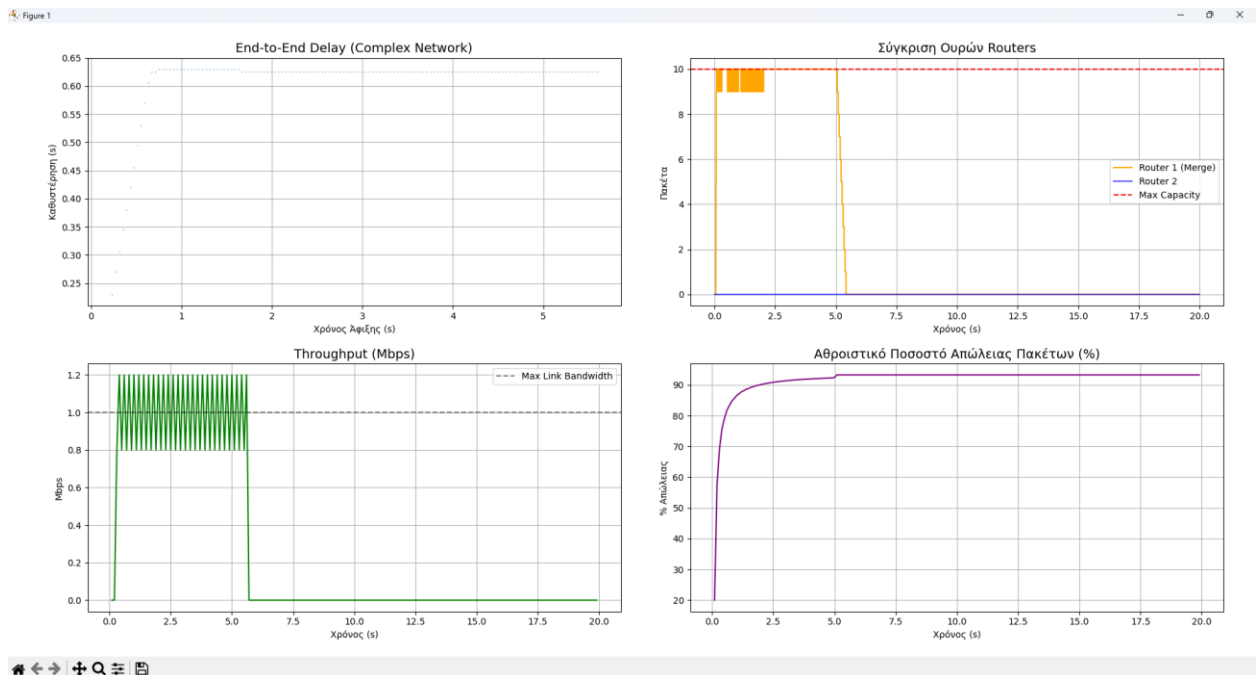
B. Μεγάλα Πακέτα (3000 Bytes)

- **Load:** $400 \times 24000 = 9.6 \text{ Mbps}$.
- **Συμπεριφορά:** Καταστροφική συμφόρηση.
- **Packet Loss:** Αγγίζει το ~89%. Ο Router 1 μπορεί να περάσει ελάχιστα πακέτα.
- **Delay:** Αυξάνεται δραματικά λόγω του μεγάλου Transmission Delay σε κάθε ένα από τα 3 hops.



Γ. Πολύ Μεγάλα Πακέτα (5000 Bytes)

- **Load:** $400 \times 40000 = 16 \text{ Mbps}$.
- **Συμπεριφορά:** Το δίκτυο καταρρέει σχεδόν πλήρως.
- **Packet Loss:** Ξεπερνά το **93%**.
- **Delay:** Κάθε πακέτο που καταφέρνει να περάσει υφίσταται τεράστια καθυστέρηση μετάδοσης σε κάθε Router.



Η επέκταση του δικτύου ανέδειξε δύο κρίσιμα συμπεράσματα:

1. **Συσσώρευση Κυκλοφορίας (Traffic Aggregation):** Σε σύνθετα δίκτυα, τα σημεία όπου ενώνονται πολλές ροές (όπως ο Router 1) γίνονται τα κύρια σημεία συμφόρησης (Hotspots). Ο επόμενος Router (R2) υπολειτουργεί επειδή ο προηγούμενος δεν μπορεί να τον τροφοδοτήσει αρκετά γρήγορα.
2. **Αυξημένη Ευαισθησία:** Το σύνθετο δίκτυο είναι πιο ευαίσθητο στον όγκο δεδομένων. Ενώ στο απλό δίκτυο τα 1000 Bytes/pkt προκαλούσαν ~37% απώλειες, εδώ (με διπλάσιες πηγές) προκαλούν ~68% απώλειες.

Ερώτημα 8:

Μεθοδολογία

Σκοπός του ερωτήματος είναι η σύγκριση δύο αλγορίθμων διαχείρισης ουράς στον κεντρικό Router (Router 1):

FCFS (First-Come-First-Serve): Τα πακέτα εξυπηρετούνται αυστηρά με τη σειρά άφιξης. Χρησιμοποιείται μία κοινή ουρά για όλους.

RR (Round-Robin): Ο Router διατηρεί ξεχωριστές ουρές για κάθε πηγή (Host A, Host B) και τις εξυπηρετεί κυκλικά (ένα πακέτο από τον A, ένα από τον B).

Για να αναδείξουμε τις διαφορές, χρησιμοποιήσαμε το σενάριο "**Mixed Traffic**" (**Ασύμμετρη Κίνηση**):

Host A: Στέλνει μικρά πακέτα (**100 Bytes**), προσομοιώνοντας ευαίσθητη κίνηση. (με ρυθμό 0.002s)(συνολικά 1000 πακέτα)

Host B: Στέλνει μεγάλα πακέτα (**3000 Bytes**), προσομοιώνοντας μεταφορά αρχείων. (με ρυθμό 0.02s)(συνολικά 1000 πακέτα)

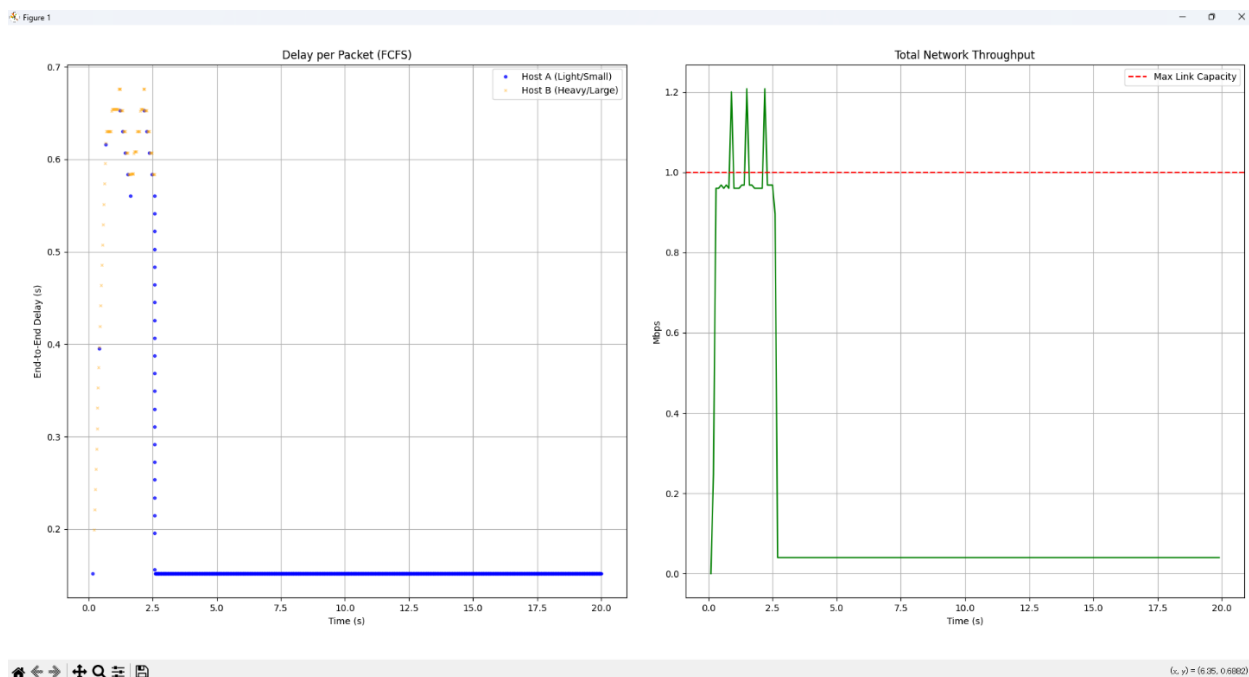
Bandwidth: 1 Mbps.

2. Ανάλυση Αποτελεσμάτων

A. Αλγόριθμος FCFS (First-Come-First-Serve)

Φαινόμενο Head-of-Line Blocking: Όταν ο Host A στέλνει ένα μικρό πακέτο, αν μπροστά του στην ουρά υπάρχει ένα μεγάλο πακέτο του Host B, το μικρό πακέτο "κολλάει". Πρέπει να περιμένει να μεταδοθούν τα 24.000 bits του μεγάλου πακέτου (24ms) πριν έρθει η σειρά του.

Delay: Παρατηρούμε ότι η καθυστέρηση των μικρών πακέτων (Host A) είναι υψηλή και ασταθής, καθώς εξαρτάται από το πόσα μεγάλα πακέτα βρίσκονται μπροστά τους. Δεν υπάρχει προστασία για τη "γρήγορη" κίνηση.

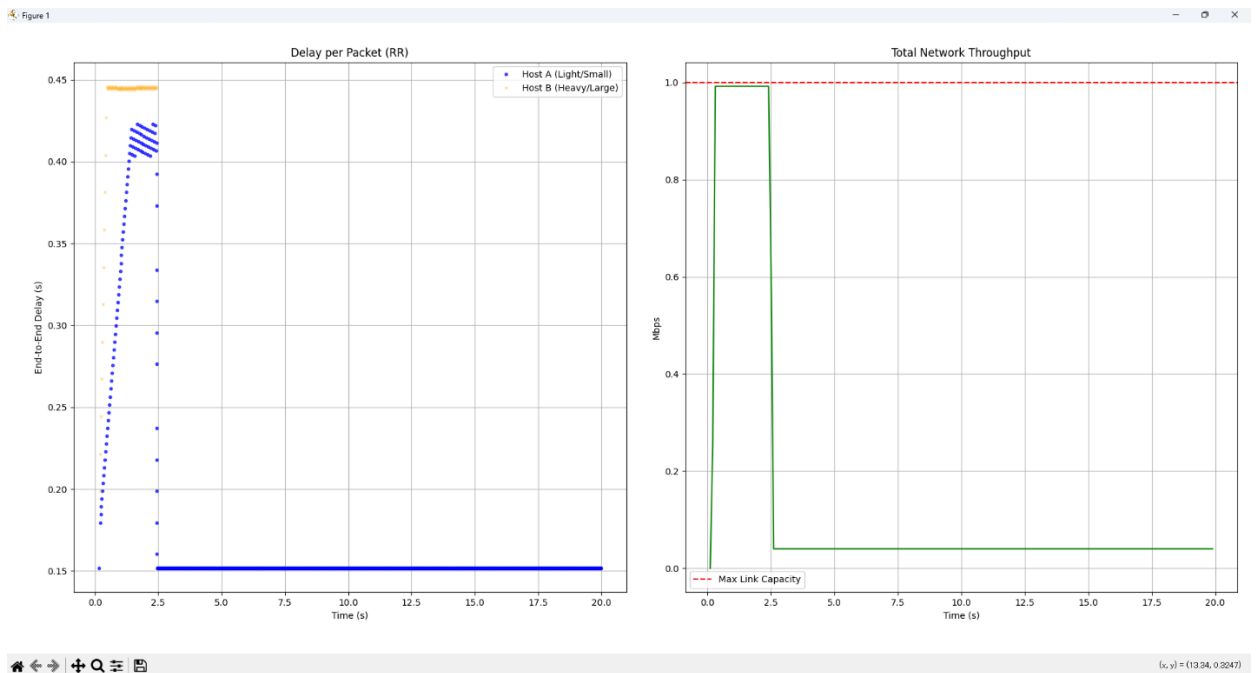


B. Αλγόριθμος RR (Round-Robin)

Δίκαιη Κατανομή (Fairness): Ο αλγόριθμος RR απομονώνει τις ροές. Ακόμα κι αν ο Host B έχει γεμίσει την ουρά του με μεγάλα πακέτα, ο Router θα εξυπηρετήσει ένα πακέτο του B και αμέσως μετά θα ελέγξει την ουρά του A.

Delay: Η καθυστέρηση για τον Host A (Small Packets) **μειώνεται δραματικά**. Τα μικρά πακέτα δεν περιμένουν πλέον πίσω από όλα τα μεγάλα, αλλά παίρνουν προτεραιότητα σε κάθε κύκλο εξυπηρέτησης.

Throughput: Το συνολικό throughput παραμένει στο όριο του 1 Mbps, αλλά η κατανομή του bandwidth είναι πιο δίκαιη ως προς τον αριθμό των πακέτων που εξυπηρετούνται.



3. Συμπεράσματα

Ο αλγόριθμος **FCFS** αποδίδει καλά όταν όλα τα πακέτα έχουν παρόμοιο μέγεθος, αλλά αποτυγχάνει σε μικτά φορτία, καθώς επιτρέπει στις βαριές ροές να μονοπωλούν τον χρόνο μετάδοσης.

Ο αλγόριθμος **Round-Robin** αποδίδει καλύτερα σε σενάρια διαφορετικών φορτίων (Mixed Load), καθώς προσφέρει απομόνωση και εξασφαλίζει ότι οι μικρές, γρήγορες ροές δεν θα πνιγούν από τις μεγάλες ροές δεδομένων.

Σας Ευχαριστούμε!